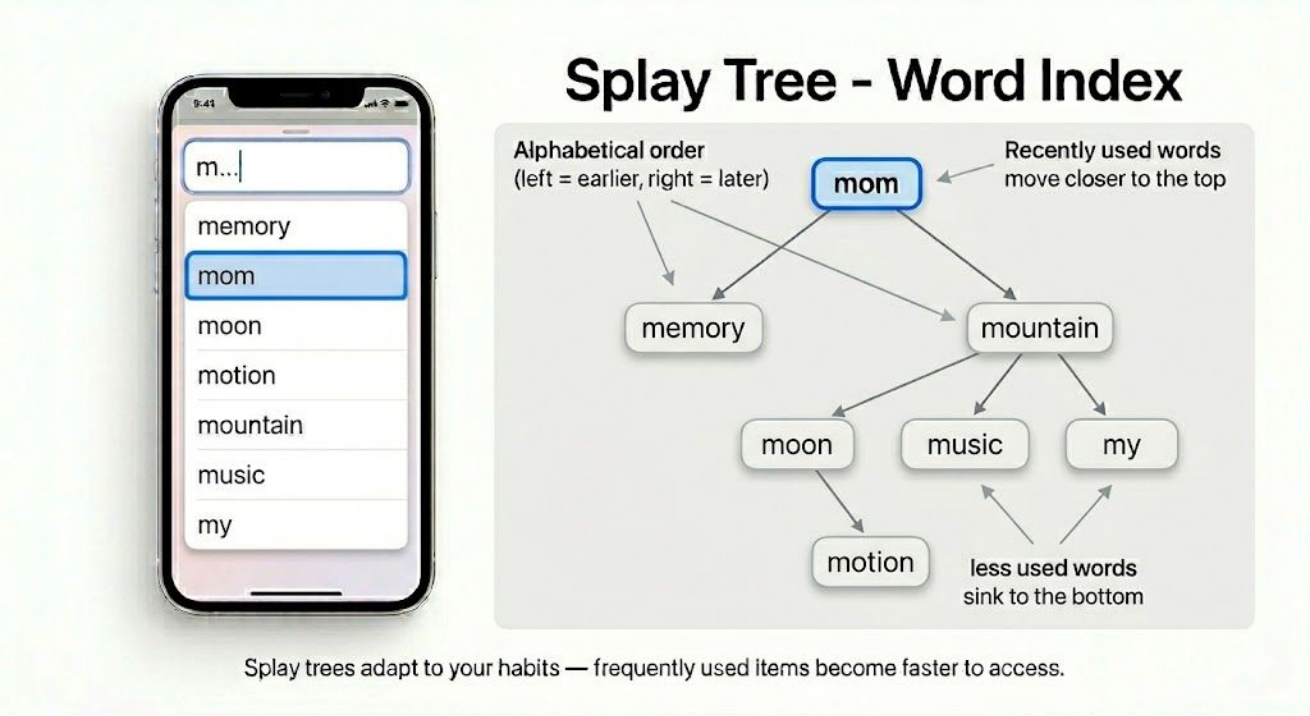
Splay Trees

Joseph Farkas, John McAlinden, Gabriel Nech,
Aesha Shah, Shivali Singh, Joshil Vaddi

Motivation

When you type on your iPhone, the phone searches a dictionary of words in alphabetical order.



Background

Time Complexity

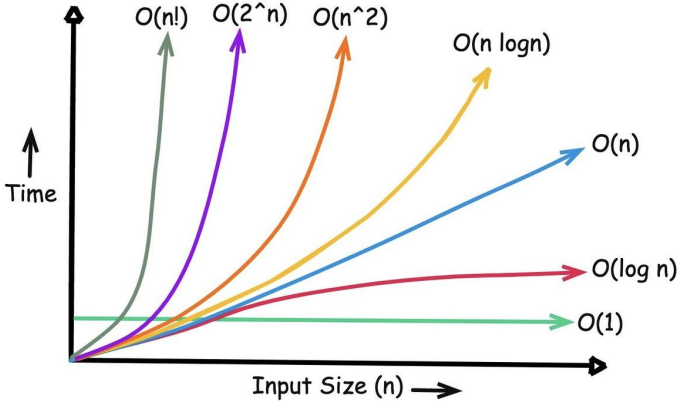
- Comparing the computational intensity of an algorithm
- Big O notation: An upper bound on how # steps grows with input size

Amortization

- Average cost per action over a sequence of actions

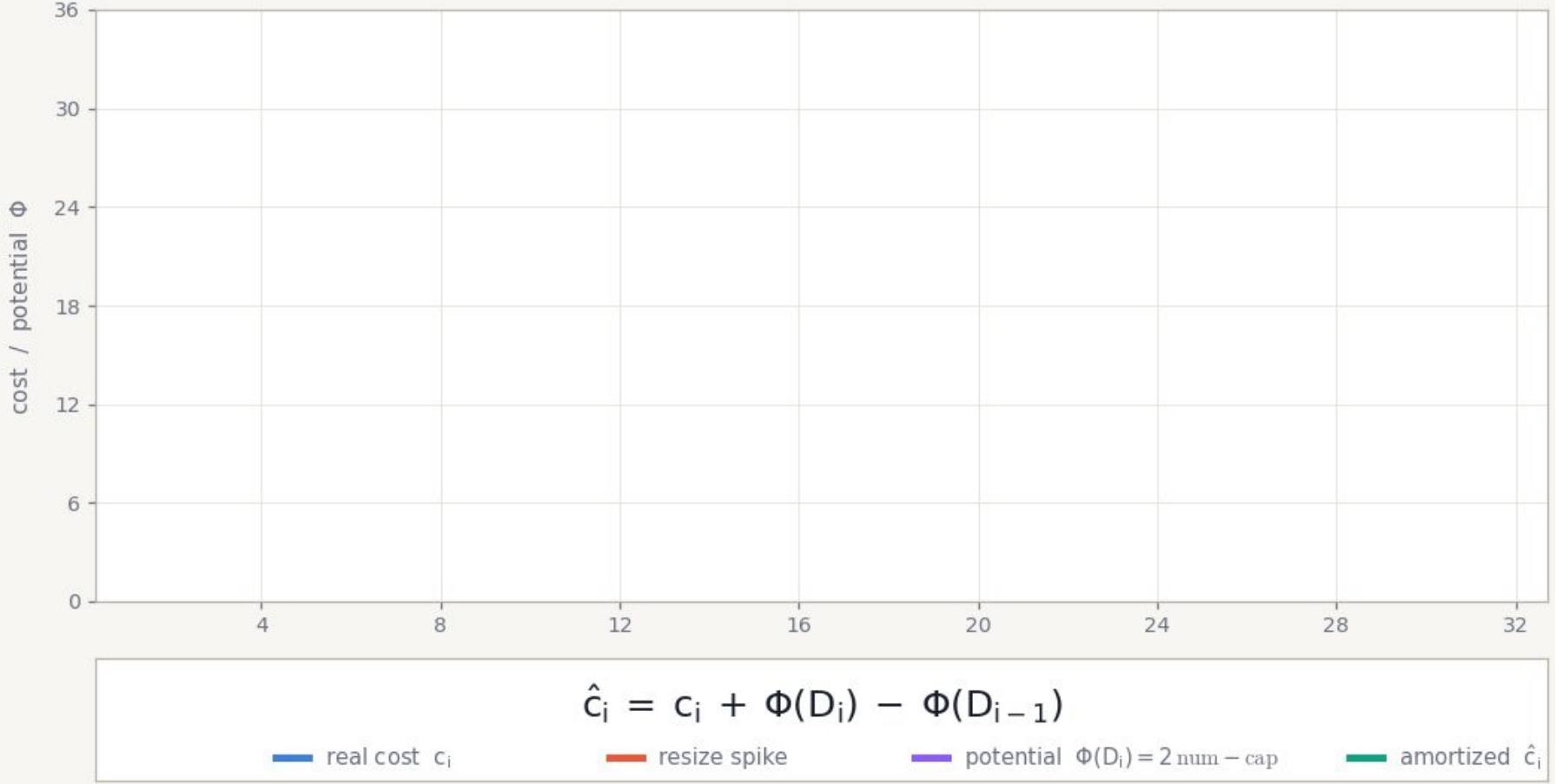
Potential

- Cheap actions “save up” for expensive actions



Amortized Analysis — The Potential Method

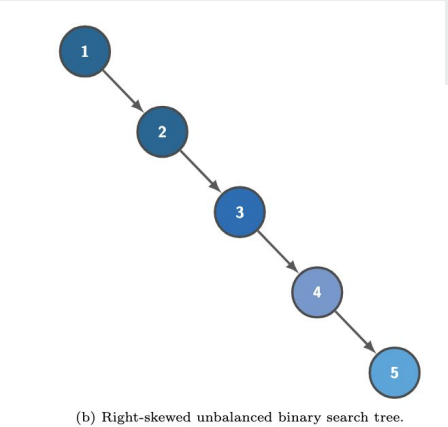
Dynamic array `append` with doubling



Background

Binary Search Trees

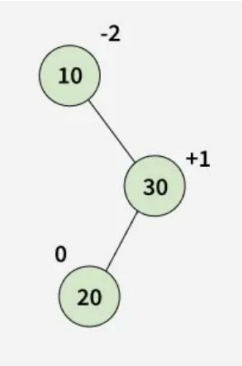
- Potentially very slow on access in degenerate case O(n).



(b) Right-skewed unbalanced binary search tree.

AVL/Red-Black

- Requires additional metadata to strictly enforce height

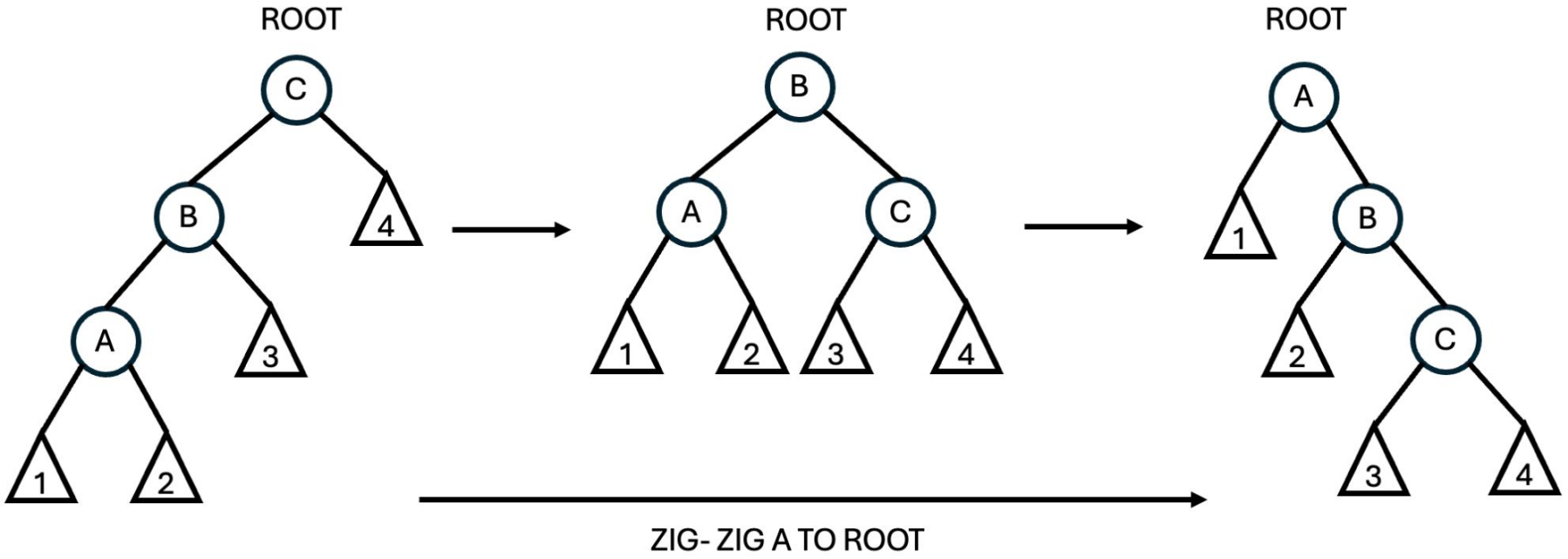


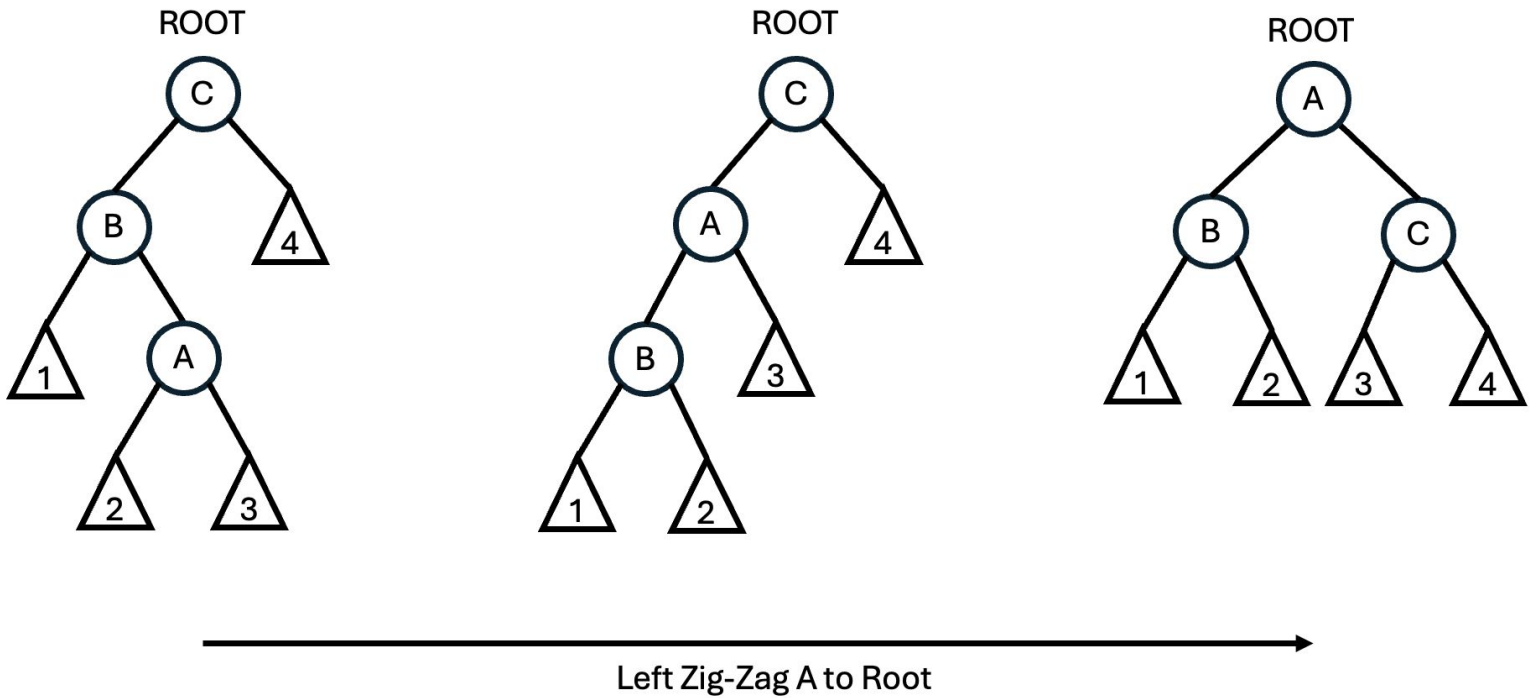
Splay Trees:

- Don’t store metadata
- Amortized O(log n) access
- Reward temporal locality


ROOT
B
A
3
1
2
ZIG A TO ROOT
ROOT
A
1
B
2
3


ROOT
B
A
3
1
2
ZIG A TO ROOT
ROOT
A
1
B
2
3


ROOT
C
B
A
4
3
1
2
ROOT
B
A
C
1
2
3
4
ROOT
A
1
B
2
C
3
4
ZIG- ZIG A TO ROOT

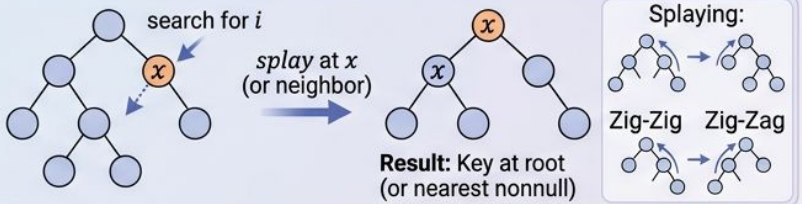

ROOT
C
B
4
1
A
2
3
ROOT
C
A
4
B
3
1
2
ROOT
A
B
C
1
2
3
4
Left Zig-Zag A to Root

Update Operations

1 I. The Foundation: Access & Splaying

$access(i, t)$ searches for i, moving the relevant node (found or nearest neighbor) to the root via splaying.

- **If not found:** Splays at the last nonnull node reached.

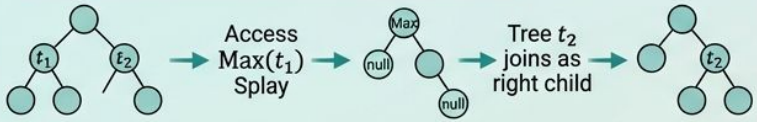


2 II. Structural Primitives: Join & Split

Key to Simplify Updates

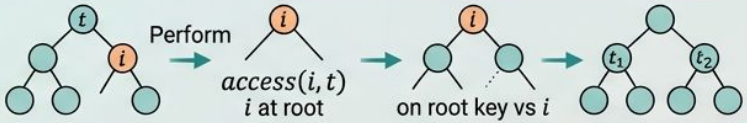
$join(t_1, t_2)$

Assumption: All items in t_1 < items in t_2



$split(i, t)$

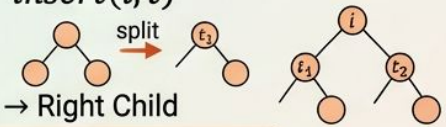
Goal: Two trees ($t_1 \le i$ and $t_2 > i$)



3 III. Update Operations (using Primitives)

$insert(i, t)$

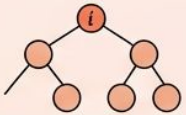
- $split(i, t) \rightarrow t_1, t_2$
- New root with i
- $t_1 \rightarrow$ Left Child, $t_2 \rightarrow$ Right Child



Alternatively: Standard search, then splay at i.

$delete(i, t)$

- $access(i, t) \rightarrow i$ at root
- Remove root
- $join$(Left Subtree, Right Subtree)



Alternatively: Find x, replace with $join$(children), splay at parent y.

Performance Guarantee

A sequence of m operations on a splay tree containing n items (starting from an empty tree).

$$O((m+n)\log(m+n))$$

Actual cost (t) of an operation j

Amortized Cost

Potential Difference

$$\sum_{j=1}^{m} t_j \leq \sum_{j=1}^{m} a_j + \Phi_0 - \Phi_m \leq O(m \log n) + n \log n$$

Practical Applications

